Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра систем штучного інтелекту



Звіт

до лабораторної роботи № 3

з дисципліни

Проектування інформаційних систем

Виконав:
студент КН-315
Лозовий О.А.

Львів – 2023

Функціональні вимоги:

1. Безпека даних
    a. Система повинна використовувати сучасні протоколи шифрування.
    b. Система має двофакторну аутентифікацію.
    c. Всі дані, які передаються та зберігаються, повинні бути зашифровані.
2. Робота з документами
    a. Система повинна автоматично перевіряти подані документи на ідентичність.
    b. Інтеграція з органами, що видають документи, для швидкої і надійної перевірки.
3. Оцінка кредитного ризику
    a. Система повинна мати алгоритми для оцінки кредитної здатності особи.
    b. Система використовуватиме машинне навчання для аналізу фінансової історії, перевірки доходів та інших показників.
    c. Система повинна використовувати наявні бази фінансових історій користувачів.
4. Взаємодія з клієнтами
    a. Надсилання нотифікацій про стан заявки.
    b. Користувач має мати можливість отримати доступ до інформації через особистий кабінет.
    c. Користувач має мати можливість отримати доступ до перегляду стану заявки, історії платежів, та іншої важливої інформації.
5. Платіжні системи
    a. Інтеграція з платіжними системами для здійснення транзакцій.
    b. Можливість оплатити різними засобами платежу.
6. Аудит та звітність
    a. Автоматична генерація звітів для внутрішнього аудиту та регуляторних органів.

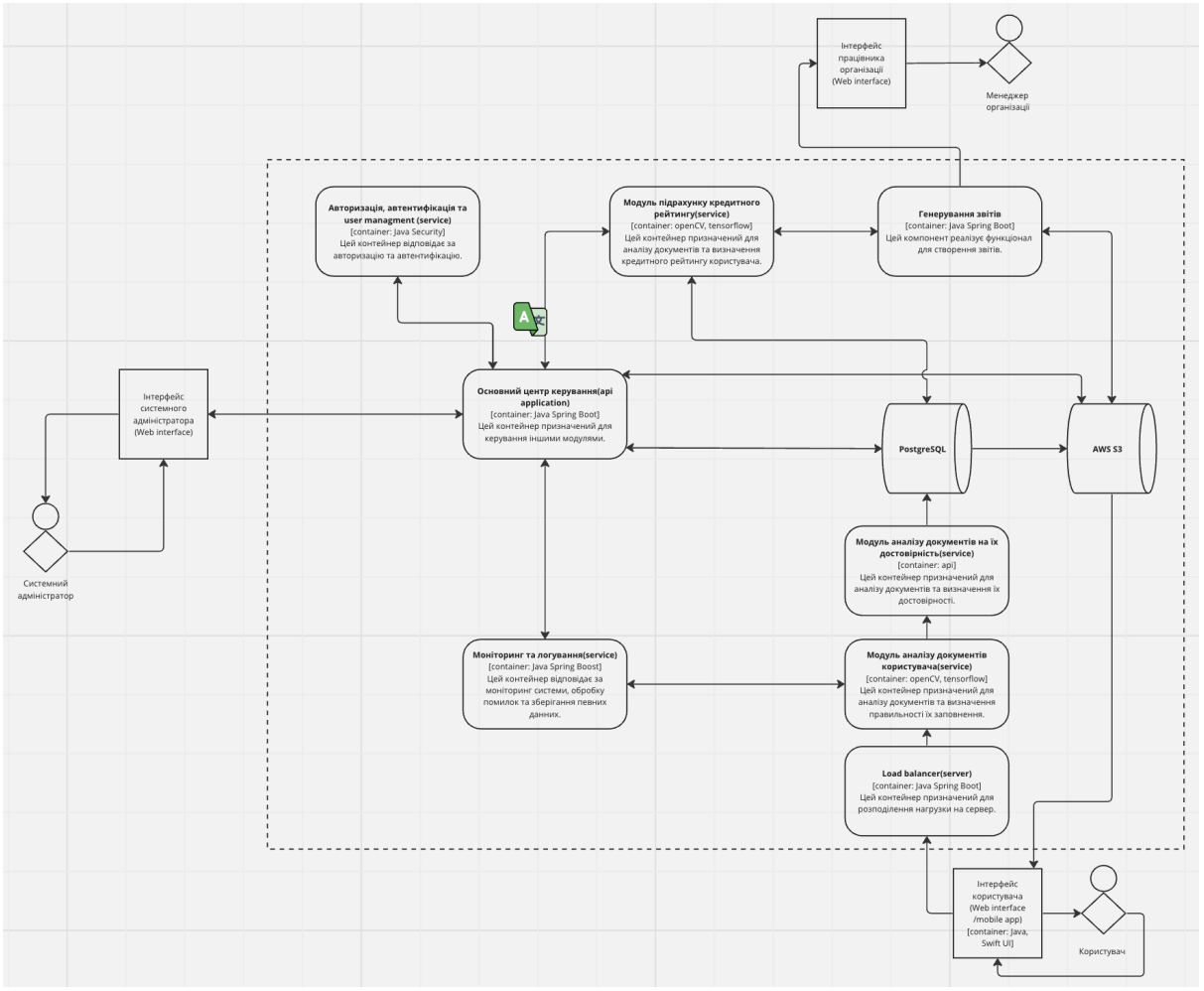
Нефункціональні вимоги:

1. Уся персональна інформація повинна зберігатись за стандартами GDPR.
2. Підтримка платежів через банківські картки, електронні гаманці та інші засоби платежу.
3. Створені звіти повинні відповідати стандартам і вимогам регуляторів.

Architecturally Significant Requirements:

1. Збереження даних
    a. Збереження звітів вимагає таку базу даних, яка легко може збільшуватись разом зі збільшенням кількості звітів.
    b. Дані користувачів також необхідно зберігати (у зашифрованому вигляді).
    c. Для зберігання резервних копій моделі(на випадок збою), необхідно виділити місце на сервері.
2. Система повинна підтримувати можливість збільшення кількості користувачів
    a. Збільшення користувачів приведе до потреби більшої бази даних, яка може вмістити більшу кількість користувачів.
    b. Також при збільшенні кількості запитів, можливі збої в роботі, тому необхідно обмежити трафік при великих навантаженнях.
3. Інтеграції
    a. Для інтеграції з органами, що видають документи, необхідно створити API, або ж інтегрувати зовнішнє API.
    b. Для інтеграції машинного навчання, необхідно створити модель, натренувати її та перевірити точність її роботи.
4. Мультиплатформність
    a. Необхідно створити мобільний додаток для IOS і Android.
    b. Необхідно створити веб-інтерфейс для інших платформ.

Діаграма контейнерів:



Стратегія тестування:

1. **Scope тестування**

   Мета тестування полягає в переконанні, що система оцінки кредитного рейтингу відповідає всім визначеним функціональним та нефункціональним вимогам. Також важливо виявити та усунути будь-які помилки або проблеми, що можуть виникнути під час її використання.

2. **Підходи, рівні, види тестування**

   Консультативні підходи з залученням замовника та експерта

Тестування системи, в якому беруть участь замовник та інші представники сторони замовника(експерт з фінансів), має на меті перевірку відповідності системи вимогам та бажаним функціональностям за участю замовника та інших зацікавлених сторін.

Види тестування:

1. Мануальне Black-box

Технології:

1. Kibana – зручний інструмент для візуалізації статистичних даних під час тестування системи
2. Postman – зручний інструмент для мануального тестування API запитів системи

Рівні тестування:

1. Тестування інтеграцій (Тестування правильності, зручності та повноцінності інтеграцій з сторонніми api)
2. Тестування системи (Тестування правильності поведінки системи згідно вимог фінансової організації)
3. Тестування прийняття (Тестування системи користувачем згідно вимог та потреб фінансової організації, дотримання усіх робочих правил та дійсного законодавства)

Метрики тестування:

1. Загальна оцінка замовником системи (від 0 до 10)
2. Загальна оцінка експерта для системи обрахунку кредитного рейтингу (від 0 до 1)

Підхід на основі ризиків

Тестування всіх елементів та складових системи у найвразливіших до ризиків областях, визначених аналітично, спрямоване на забезпечення найвищого рівня безпеки та стабільності системи.
Види тестування:

1. Автоматизоване White-box
2. Автоматизоване Black-box
3. Мануальне Black-box

Технології:

1. JUnit - це найбільш ефективний та популярний фреймворк для тестування функціоналу, написаного мовою Java.
2. XCUI - вважається найефективнішим і найпопулярнішим фреймворком для тестування функціоналу, реалізованого мовою Swift.
3. PyTest - це популярний фреймворк, який ефективно працює з Django і використовується для тестування сервісів, написаного мовою Python.
4. Postman – інструмент, що дозволяє тестувати API запити системи і опісля автоматизувати їх тестування

Рівні тестування:

1. Тестування компонент (Тестування малих частин системи, таких як функції, класи, внутрішня логіка та інше, має на меті перевірку їх правильності, відповідність виконанню завдань, відсутність багів та інших недоліків. Це також включає тестування безпекових аспектів системи, зокрема перевірку правильності та стійкості відносно потенційних загроз та вразливостей.)

2. Тестування інтеграцій (Тестування взаємодії системи зі сторонньою базою даних їх доступністю та правильністю, а також з дотриманням безпеки та стандартів захисту)
3. Тестування системи (Тестування поведінки системи, роботи з документами, тощо, на їх правильність, безвідмовність та працездатність, а також захищеність)
4. Тестування прийняття (Тестування системи з погляду кінцевого користувача, його зручності, розуміння та потреб)

Метрики тестування:

1. Відсоток покритого тестами коду (%)
2. К-сть знайдених помилок (число)
3. Відсоток тестів (%)
    a. пройдених
    b. непройдених
    c. пропущених
4. Загальна оцінка захищеності системи та її підсистем (від 0 до 2)
5. Загальна оцінка користувачем системи (від 1 до 10)

Модельний підхід на основі відмов

Тестування системи в областях потенційних відмов та під впливом найвищих навантажень, які визначаються на основі створеної моделі системи, з використанням математичних та статистичних методів, спрямоване на перевірку надійності та ефективності системи в умовах максимального напруження.

Види тестування:

1. Автоматизоване Black-box
2. Мануальне Black-box

3. Автоматизоване White-box
4. Мануальне White-box

Технології:

1. JUnit - це найбільш ефективний та популярний фреймворк для тестування функціоналу, написаного мовою Java.
2. XCUI - вважається найефективнішим і найпопулярнішим фреймворком для тестування функціоналу, реалізованого мовою Swift.
3. PyTest - це популярний фреймворк, який ефективно працює з Django і використовується для тестування сервісів, написаного мовою Python.
4. JMeter - потужний інструмент для проведення навантажу та стрес-тестування систем.
5. Kibana - зручний та багатофункціональний інструмент для візуалізації статистичних даних під час тестування системи.

Рівні тестування:

1. Тестування компонент (Тестування малих частин системи, на їх потенційну відмовність та час виконання при великих навантаженнях)
2. Тестування інтеграцій (Тестування взаємодії системи зі стороннім арі за великого навантаження арі та/або власної системи)
3. Тестування системи (Тестування поведінки системи за великого навантаження)
4. Тестування прийняття (Тестування системи кінцевим користувачем за великого навантаження, перевірка комфортну та зручності керування такою системою)

Метрики тестування:

1. Відсоток правильності обробки найпоширеніших запитів (%)
2. Середній час обробки запиту до стороннього api за малого навантаження (секунди)
3. Середній час обробки запиту до стороннього api за великого навантаження (секунди)
4. Середній час роботи складових системи за великих навантажень (секунди)
5. Відсоток відхилених/прийнятих запитів за малого навантаження (%)
6. Відсоток відхилених/прийнятих запитів за великого навантаження (%)
7. Загальна оцінка користувачем системи за великого навантаження (від 1 до 10)

3. Заплановані Quality Gates

Для забезпечення якості системи такі Quality Gates:

1. Coverage Gate: Вимірювання покриття коду тестами - допомагає переконатися, що тестові сценарії покривають достатню частину коду. (80% покриття коду та 80% покриття розгалужень if… else…)
2. Performance Gate: Встановлення максимального допустимого часу відповіді системи на запити допомагає переконатися, що продукт працює на прийнятному рівні продуктивності. (5 секунд на запити авторизації, реєстрації та отримання даних, 30 секунд на запити відправлення даних)
3. Integration Gate: Виконання автоматизованих інтеграційних тестів та перевірка ефективності роботи з інтегрованими системами, для перевірки взаємодії між різними компонентами системи та для підтвердження правильності їх роботи під час спільної опрацювання даних та запитів. (80% вдалих запитів та 80% пройдених тестів)

4. Stability Gate: Моніторинг частоти виявлення помилок при виконанні автоматизованих тестів для контролю стабільності та функціональності системи. (90% пройдених тестів та 0 критичних помилок)
5. Security Gate: Автоматизоване тестування безпеки системи забезпечує відповідність системи стандартам безпеки шляхом виявлення та усунення потенційних проблем безпеки. Це досягається за допомогою сканування вразливостей та аналізу вразливостей за допомогою спеціалізованих інструментів. (98% пройдених тестів)
6. Acceptance Gate: Забезпечення виконання всіх вимог та потреб користувачів. (8.0 - середня оцінка менеджерів організації, 8.0 - середня оцінка клієнтів фін. організації, не менше 5.0 - для будь якого пункту)

Тест – план для системи генерації звітів:

Загальний опис функціоналу тестування:

Тест-план орієнтований на перевірку функціональності системи генерації звітів, зокрема з урахуванням їх надсилання менеджеру організації.

Тест-кейс 1: генерування звіту

Передумова: отримано дані з про Тестового користувача та його кредитний рейтинг

Крок 1: згенерувати звіт про Тестового користувача

Крок 2: отримати звіт про Тестового користувача

Крок 3: перевірити звіт на наявність відмінностей від Дійсного звіту

Результат: Звіти збігаються (Похибка становить не більше 1%).

Тест-кейс 2: отримання звіту про клієнта менеджером організації

Передумова: менеджер знаходиться на сторінці отримання звітів

Крок 1: Обрати необхідного клієнта

Крок 2: Натиснути правою кнопкою мишу на нього

Крок 3: Обрати пункт «Отримати звіт»

Результат: менеджер отримує звіт про клієнта.

Тест-кейс 3: отримання звіту про клієнта НЕ менеджером організації

Передумова: користувач знаходиться на сторінці отримання звітів

Крок 1: Обрати необхідного клієнта

Крок 2: Натиснути правою кнопкою мишу на нього

Крок 3: Обрати пункт «Отримати звіт»

Результат: користувач не отримує звіт про клієнта, користувач отримує повідомлення про помилку доступу, робиться відповідний запис в error log.

Тест-кейс 4: проверка на коректність отриманих данних

Передумова: отримано некоректний набір даних

Крок 1: введення некоректних даних користувача, некоректний формат кредитного рейтингу, тощо

Крок 2: перевірити, чи система виводить коректні повідомлення про помилки

Результат: отримано коректне повідомлення про помилку, робиться відповідний запис в error log.

Тест-кейс 5: неможливість отримати доступ до даних користувача без авторизації та відповідної ролі

Передумова: Користувач авторизований та знаходиться у секції документів

Крок 1: Намагатися отримати несанкціонований доступ до звітів інших користувачів.

Крок 2: Перевірити, чи система ефективно захищена від несанкціонованого доступу та зберігає конфіденційні дані відповідно до стандартів безпеки.

Результат: доступ не отримано

Висновок: На цьому етапі лабораторної роботи я розробив стратегію тестування для системи підрахунку кредитного рейтингу, виходячи з уважного аналізу вимог і структури системи. У рамках стратегії я докладно описав цілі, підходи, метрики та інструменти. Також був розроблений план тестування, який включає кейси для основного компонента генерації звітів.